Makerere Campus Route Finder: BFS and DFS

CSC 2114 Artificial Intelligence, Practical 1 | Name: _____ | Student No.: _____ | Code: `route_finder.py` (Python 3, standard library only)

1. Problem formulation

Let **P** be the set of 18 places (14 schools, 4 landmarks) and **E** the set of 24 two-way connections. The state space is **S = P**: a state is simply the place where the visitor currently stands.

Component	Definition	Type
1. Initial state	$s_0 \in S$, the source school typed by the user, e.g. s_0 = CoCIS.	One element of the set S
2. Actions(*s*)	Actions(s) = { Go(x) : {s, x} ∈ E }. Read off the graph by taking every edge that contains *s in either position* (connections are two-way); the other end is a legal destination. The code builds this once as an adjacency list and sorts it alphabetically. Example: Actions(Main Library) = {Go(Freedom Square), Go(Main Building), Go(School of Economics), Go(School of Education)}.	Function S → set of actions
3. Result(*s*, *a*)	Result(s, Go(x)) = x, defined only for Go(x) ∈ Actions(s). Walking to a neighbour puts the visitor at that neighbour; the model is deterministic.	Function S × A → S
4. Goal test	IsGoal(s) = (s = g), where g is the destination school. There is exactly one goal state.	Boolean test on a state
5. Path cost	g(path) = Σ c(s, a, s') with c = 1 for every step, so the path cost is the number of hops. The step cost is 1 because the model chooses to measure a route by the *number of walks between adjacent places* and deliberately ignores physical distance; every action is therefore equally expensive and the optimal route is the one with the fewest hops.	Function path → non-negative integer

Abstraction. The state ignores (1) the physical length and walking time of each connection, (2) terrain such as the slope up to Main Building, stairs and footpath quality, and (3) conditions such as rain, crowds, traffic on campus roads and the time of day. Dropping them is *valid* because every abstract step is a real walkable connection, so any abstract route can be expanded into a real walk the visitor can actually make. It is *useful* because it reduces a continuous, changing world to 18 states and 24 edges, which can be searched instantly and whose answer is a short, readable list of places.

State versus node. The state *Main Library* can appear in two different search-tree nodes: one reached by Main Building → Main Library (PARENT = Main Building, PATH-COST = 1) and one by Main Building → Freedom Square → Main Library (PARENT = Freedom Square, PATH-COST = 2). The STATE is the same, but PARENT, ACTION and PATH-COST differ. My program shows this happening: in DFS from CoCIS, the state CoCIS is the root node (cost 0) and appears again as a child of Central Teaching Facility (cost 2) (see the trace in §4.2).

2. Design

- **Graph and problem.** `build_graph(EDGES)` adds each edge in both directions and stores every place's neighbours sorted alphabetically. `RouteProblem` holds the five components: `initial`, `actions(s)`, `result(s, a)`, `is_goal(s)` and `step_cost` (always 1).
- **Node.** A class with the four lecture fields: `state`, `parent`, `action` and `path_cost`. `solution(node)` follows `parent` pointers to the root, collecting states, then reverses the list.
- **reached.** A `dict` mapping state → node. A child is recorded when it is *generated*, and later children with a state already in `reached` are skipped. As in lecture step 1, `reached` starts empty, so the start state can be generated once more as a grandchild. This costs at most one extra expansion, and I kept it to follow the lecture exactly.
- **Frontier: the only difference between BFS and DFS.** Two small classes share one interface, `add_all(nodes)` and `remove()`. `FIFOQueue` uses a `deque`, adding at the back and removing from the front. `LIFOStack` uses a `list`, adding at the back and removing from the back. Because a stack returns the *last* node pushed, `LIFOStack.add_all` pushes the children in *reverse* alphabetical order so that the alphabetically first child ends up on top and is explored first. `search()` chooses the class from `STRATEGIES[strategy]`. No other line depends on the algorithm.
- **Switches.** `search(..., use_reached=False)` or the `--tree` command-line flag gives tree-like search. A cap of 10,000 expansions makes every run terminate.

```
function SEARCH(problem, strategy, use_reached):
  frontier ← new FIFOQueue if strategy = BFS else new LIFOStack     # the only difference
  frontier.add(NODE(problem.initial)); reached ← {}; expanded ← 0       # step 1
  while frontier is not empty:                                          # steps 2 and 7
      node ← frontier.remove()                                          # step 3
```

```
    if problem.IS-GOAL(node.state): return SOLUTION(node), expanded   # step 4 (goal test on removal)
    if expanded = 10,000: return CUTOFF
    expanded ← expanded + 1                                           # step 5: expand
    children ← []
    for each action in problem.ACTIONS(node.state):   (alphabetical)
        child ← NODE(RESULT(node.state, action), node, action, node.path_cost + 1)
        if use_reached:                                               # step 6
            if child.state in reached: continue
            reached[child.state] ← child
        children.append(child)
    frontier.add_all(children)        # LIFOStack pushes in reverse so the first child is on top
return FAILURE                                                        # step 2
```

Input handling: the 14 schools are listed with numbers 1–14, and the user may type either the number or the name. Names are compared after `strip()` and `casefold()`; a number outside 1–14 is rejected with a message. An unknown place or a landmark entered as a school prints a clear message, lists the 14 valid schools and asks again. An unknown algorithm asks again. Source = destination prints a one-place route with cost 0 and 0 expansions. Failure prints `No route found.`

B4. Graph search vs. tree-like search (DFS, CoCIS → School of Law)

Mode	Outcome	Cost	Nodes expanded	Stack size at end
With `reached` (graph search)	CoCIS → Central Teaching Facility → Freedom Square → School of Law	3	16	≤ 9
Without `reached` (`--tree`)	No route found. Stopped at the safety cap.	–	10,000	20,001

Why the cycles cause this. CoCIS's alphabetically first neighbour is Central Teaching Facility (CTF), and CTF's first neighbour is CoCIS. Without `reached`, nothing stops DFS from regenerating CoCIS as a child of CTF, so it always takes the top branch: CoCIS → CTF → CoCIS → CTF → …. The search tree is infinite even though there are only 18 states, because every cycle can be walked around again. Each expansion also leaves two unexplored siblings below it on the stack (net +2 per expansion), so after 10,000 expansions the stack holds 20,001 nodes and Freedom Square, the way to Law, is never reached. **How `reached` prevents it:** each state is put on the frontier at most once (plus the start state at most once more), so DFS can expand at most about 19 nodes, cannot loop, and is complete on this finite graph. With the same flag, tree-like BFS still finds the 3-hop route (19 expansions) because it goes level by level, but its frontier grows to 41 nodes instead of at most 8.

3. Results (graph search, alphabetical neighbour order)

Key: MB Main Building · FS Freedom Square · ML Main Library · CTF Central Teaching Facility · CoCIS · ENG Engineering · SBE Built Environment · MT Margaret Trowell School of Industrial and Fine Art · LAW Law · EDU Education · SSS Social Sciences · LPA Liberal and Performing Arts · ECON Economics · SSP Statistics and Planning · CoNAS College of Natural Sciences · FOOD Food Technology, Nutrition and Bioengineering · AGRIC Agricultural Sciences · VET Veterinary Medicine. Full names are used by the program.

#	Source	Destination	Alg.	Route	Cost (hops)	Nodes expanded
1	ECON	LAW	BFS	ECON → LAW	1	2
1	ECON	LAW	DFS	ECON → LAW	1	17
2	CoCIS	ENG	BFS	CoCIS → ENG	1	2
2	CoCIS	ENG	DFS	CoCIS → ENG	1	17
3	LAW	SSS	BFS	LAW → FS → MB → SSS	3	10
3	LAW	SSS	DFS	LAW → FS → MB → SSS	3	10
4	EDU	VET	BFS	EDU → CoNAS → FOOD → AGRIC → VET	4	14
4	EDU	VET	DFS	EDU → CoNAS → FOOD → AGRIC → VET	4	5
5	SSP	LAW	BFS	SSP → ECON → LAW	2	8
5	SSP	LAW	DFS	SSP → CTF → FS → LAW	3	15
6	CoCIS	ECON	BFS	CoCIS → SSP → ECON	2	8
6	CoCIS	ECON	DFS	CoCIS → CTF → FS → ML → ECON	4	15
7	LAW	MT	BFS	LAW → FS → MB → LPA → MT	4	13
7	LAW	MT	DFS	LAW → FS → CTF → CoCIS → ENG → MT	5	5
8	CoCIS	SBE	BFS	CoCIS → ENG → SBE	2	7
8	CoCIS	SBE	DFS	CoCIS → CTF → FS → MB → LPA → MT → SBE	6	7
9	AGRIC	LPA	BFS	AGRIC → FOOD → CoNAS → EDU → ML → MB → LPA	6	13
9	AGRIC	LPA	DFS	AGRIC → FOOD → CoNAS → EDU → ML → FS → CTF → CoCIS → ENG → MT → LPA	10	10
10	ENG	VET	BFS	ENG → CoCIS → CTF → FS → ML → EDU → CoNAS → FOOD → AGRIC → VET	9	18
10	ENG	VET	DFS	ENG → CoCIS → CTF → FS → MB → SSS → EDU → CoNAS → FOOD → AGRIC → VET	10	11

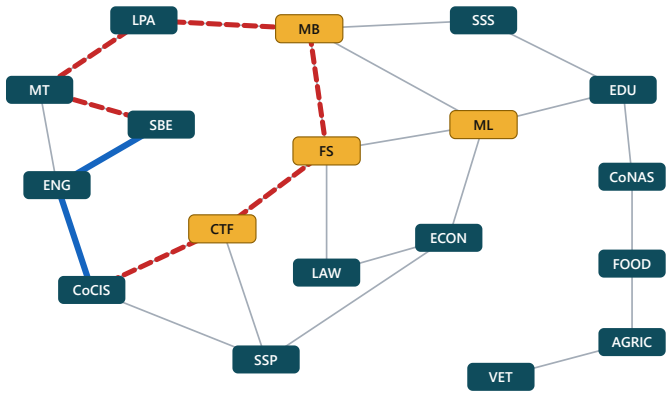
Conditions: BFS and DFS return different routes in pairs 5–10 (six pairs). Pairs 1 and 2 are directly connected schools. Pairs 9 and 10 are at least six hops apart (6 and 9 hops by BFS).

4. Analysis

4.1 Why BFS always returns a fewest-hops route

In all ten pairs, BFS's cost is less than or equal to DFS's. To check more widely, I compared BFS against true shortest distances (all-pairs Floyd–Warshall) for all **182** ordered school pairs, and BFS was optimal in **182/182**. The reason is that a FIFO queue removes nodes in order of depth: every node at depth d leaves the frontier before any node at depth d+1, so the first goal node removed is the shallowest one. The condition from the lecture is that **BFS is cost-optimal when all actions have the same cost**, so that path cost is depth (a non-decreasing function of depth). That holds here because every hop costs 1. The `reached` table does not break this: under FIFO order, the first time a state is generated is already at its minimum depth, so skipping later copies never throws away a shorter path.

4.2 A pair where DFS returns a longer route: CoCIS → School of the Built Environment (row 8)



Solid blue: BFS route, 2 hops. Dashed red: DFS route, 6 hops. Amber boxes are landmarks.

BFS expands CoCIS, CTF, ENG, SSP, CoCIS, FS, MT and then removes SBE (depth 2): **2 hops**.

DFS expands CoCIS, CTF, CoCIS, FS, MB, LPA, MT and then removes SBE: **6 hops**.

Why DFS stopped at the longer route. CoCIS's children are [CTF, ENG, SSP] in alphabetical order, so CTF is on top of the stack. DFS always continues from the most recently generated node and follows the first unreached neighbour each time: CTF → FS → MB → LPA → MT. ENG, which is one step from the goal, was generated at the first step and stored in `reached`, but it sits at the bottom of the stack and is never expanded, so SBE is never generated from ENG. When MT is expanded, SBE is not in `reached` yet, so it is pushed, removed and passes the goal test. DFS returns the *first* goal node it removes. It has no notion of depth or cost, so it does not check whether a shorter path exists.

4.3 Nodes expanded: is DFS always cheaper?

No. DFS wins when the alphabetically first branch leads to the goal: Law → MT (5 vs. 13) and Education → Vet (5 vs. 14). It loses badly on **Economics → Law (17 vs. 2)**, although the two schools are directly connected. Economics' neighbours are [Main Library, Law, Statistics and Planning], so Main Library is popped first and DFS sinks into the rest of the campus. Law is already in `reached`, so it is never regenerated nearer the top of the stack. Because the goal test happens on removal, the Law node waits at the bottom while DFS works through nearly the whole campus (17 expansions). CoCIS → Engineering (17 vs. 2) and CoCIS → Economics (15 vs. 8) behave the same way, and pairs 3 and 8 are ties. Over all 182 ordered pairs, DFS expanded fewer nodes in 110, the same number in 19, and **more in 53**. DFS saves work only when its arbitrary (here alphabetical) choice happens to point towards the goal.

4.4 The memory trade-off

BFS must store the whole current layer of the tree, so its frontier grows as $O(b^d)$. DFS stores only the current path plus the unexplored siblings along it, $O(bm)$. In this graph, BFS's frontier grows at the **hubs**: the landmarks Main Building, Freedom Square and Main Library each have degree 4. It also grows at the **cycles** (the MB–FS–ML triangle, the CTF–CoCIS–SSP and ENG–SBE–MT triangles and the long outer loops), which create many paths of equal length to the same place. With `reached`, the graph is so small that BFS's frontier never exceeded 8 nodes in any of the 182 runs. Without `reached`, every cycle multiplies the number of paths: tree-like BFS from Engineering to Veterinary Medicine (9 hops) expanded 9,043 nodes and held **18,435 nodes in the frontier at once**. Time and memory both grow as b^d, but time only accumulates, whereas every frontier node has to be held in memory at the same moment. A machine can be left running for hours but cannot hold terabytes of nodes. In Russell & Norvig's example (b = 10, depth 10), BFS needs about 3 hours but about 10 terabytes of memory. That is why the lecture says memory, not time, limits BFS in practice.

4.5 A limitation of the equal-cost assumption

Counting hops treats a short step such as Main Building → Freedom Square the same as a long walk such as Agricultural Sciences → Veterinary Medicine. BFS therefore minimises the *number of segments*, not the distance or walking time. For example, in row 7 the 5-hop DFS route through CTF, CoCIS and Engineering could easily be shorter on the ground than BFS's 4-hop route over the hill through Main Building. If edges carried real walking distances, BFS would no longer be optimal. I would use **uniform-cost search**, the best-first search with $f(n) = g(n)$, which uses a priority queue ordered by path cost and

applies the goal test on removal (as this design already does). It is optimal for any positive step costs. With map coordinates, A* with a straight-line-distance heuristic would find the same optimal route while expanding fewer nodes.

5. Reflection

I would replace the unit cost on every edge with the **measured walking time** of each connection, including slope, so that the two directions of a hill path cost different amounts (walking up to Main Building is slower than walking down). This one change makes "optimal" mean what a visitor actually cares about, which is arriving soonest. It also turns the problem into a real test of uniform-cost search rather than BFS.